

AUTOMATION CONTROL MANUAL

ANSIBLE

AUTOMATION MANUAL

From Remote Execution to Declarative Infrastructure — **Foundations**
Full chapter detail — rendered from content Markdown

VOL 1 — FOUNDATIONS
this book

VOL 2 — ENGINEERING

VOL 3 — PRODUCTION

TABLE OF CONTENTS

01	Why Automation	03	04	First Playbooks	16
02	Agentless Architecture	07	05	Modules and Idempotency	20
03	Inventory and Targeting	11	A	Cheatsheet	24

[HOW TO READ]

Chapters render the full content Markdown: problem, architecture, targets, playbooks, idempotency, labs, interview checks.

Version anchor: Pinned to ansible-core 2.16+ with the community package; FQCN in every example..

The anchor lane – reused every chapter



WHY AUTOMATION BEATS SCRIPTS

▶ WHAT – The automation problem

Manual operation scales linearly with machines and fails quietly. Shell scripts capture **commands** but not **intent**: which machines, what state, what if state already exists, how to verify, how to retry. Configuration drifts. Knowledge lives in runbooks and heads.

MANUAL LOOP VS DECLARATIVE CONVERGE

MANUAL

Operator → SSH for-loop → hosts one by one → drift accumulates → re-run repeats side effects

DECLARATIVE

Playbook → control node → inventory select → compare desired vs current → CHANGED or OK, audited

Ansible addresses this by declaring **desired state** for **selected targets** and converging each target toward that state, reporting whether change was needed.

▶ WHY – Why Ansible exists

Problem it solves: repeatable, auditable, idempotent configuration and orchestration without per-host agents or custom orchestration code for simple cases.

Where it fits:

Provisioning (Terraform) → Configuration (Ansible) → Orchestration (K8s) → Verification (Observability)

Ansible owns the configuration layer: OS, packages, files, services, users, application config — on bare metal, VMs, cloud instances, network devices where supported.

WHY AUTOMATION BEATS SCRIPTS

▶ WHERE DOES IT FIT – Ecosystem lane

- Script (Bash): "run this command again" → no convergence check
- Image baking (Packer): snapshot state → not day-2 config
- Provisioner (Terraform): creates infrastructure → hands off to Ansible for inside-OS config
- Orchestrator (K8s): schedules containers → Ansible can provision/configure K8s nodes themselves

Decision tree (preview, detailed in PDF3):

```
WHAT ARE YOU AUTOMATING? → Infrastructure → Terraform/OpenTofu
                             → Host/app config → Ansible
```

Same question, three answers — the comparison that decides the tool:

QUESTION	SHELL LOOP	ANSIBLE
Which machines?	Hosts hardcoded in the loop	Inventory selects by group and pattern
Already correct?	Runs again blindly	Reports OK, changes nothing
Proof it worked?	Exit code of the last host	Per-host recap, every host
Repeat next week?	Hope nothing changed	Converge again with the same file

WHY AUTOMATION BEATS SCRIPTS

▶ TERRAFORM VS ANSIBLE – Provision vs configure

Terraform **provisions** infrastructure (creates it: VPCs, VMs, DNS, load balancers) and tracks it in a state file. Ansible **configures** what already exists (OS, packages, files, services, users) and converges it on every run, keeping no state of its own.

ASPECT	TERRAFORM / OPENTOFU	ANSIBLE
Owns	Infrastructure lifecycle (create, change, destroy)	Inside-OS desired state (configure, converge)
State	State file (terraform.tfstate) maps config to real resources	Stateless; compares desired vs current on the host each run
Runs	plan previews infra diff; apply changes infra	--check --diff previews config diff; run converges hosts
Re-run cost	Re-plans against APIs; safe but slow	Re-converges; OK where already correct, fast
Secrets	Provider creds + sensitive outputs	Vault-encrypted vars, no_log, become scoping

Combined workflow (the normal production shape):

```
Terraform apply → outputs IPs/hostnames → generated inventory → Ansible configures OS/apps → services verified
```

```
terraform output -json > tf-out.json    # ips of new VMs
ansible-playbook -i inventories/prod site.yml
```

Handoff rule: Terraform hands **addresses**, Ansible takes **responsibility inside the machine**. Generate inventory from Terraform outputs (dynamic inventory plugin or terraform output templated into hosts.yml) — never hand-copy IPs between tools.

Anti-patterns:

- Creating VPCs/VMs with Ansible modules → duplicates Terraform state, two sources of truth
- Configuring apps with Terraform provisioner blocks beyond bootstrap → provisioners run once at create time, never converge drift; hand off to Ansible after boot
- Storing Terraform outputs with secrets in plain files → mark sensitive = true, feed Ansible via Vault

WHY AUTOMATION BEATS SCRIPTS

▶ HOW IT WORKS – High-level loop

```
USER → ansible-playbook → CONTROL NODE (inventory, vars, tasks) → SSH/WinRM → MANAGED NODE (module)
→ TARGET STATE → RESULT (OK/CHANGED/FAILED)
```

▶ PRACTICAL – Not yet executable (intentional)

This chapter builds motivation. Execution begins in Ch 02–03. No commands to mis-run yet.

▶ WHAT CAN FAIL – Misunderstanding automation

SYMPTOM	LIKELY CAUSE	PREVENTION
Automation reintroduces drift	Scripts without idempotency	Teach CHANGED vs OK early (Ch 05)
Over-automation of infra as config	Using Ansible to create VPCs	Handoff callout: provision vs configure

▶ REAL-WORLD CONNECTION

SRE uses Ansible to converge fleet after patch window: same playbook against staging then production, diff mode first, check mode for approval.

▶ Interview check

Interview Why not just run shell scripts via SSH loop?

Scripts describe commands; automation must describe targets, desired state, convergence check, error handling, and audit result. Ansible's model adds selection, declaration, and verification around the transport that a loop lacks.

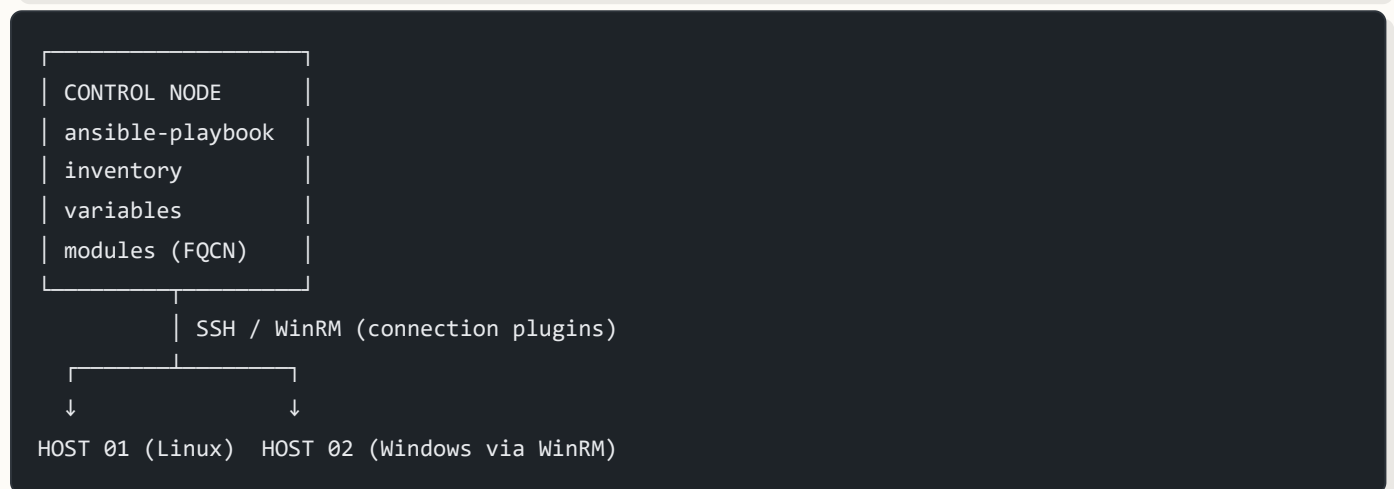
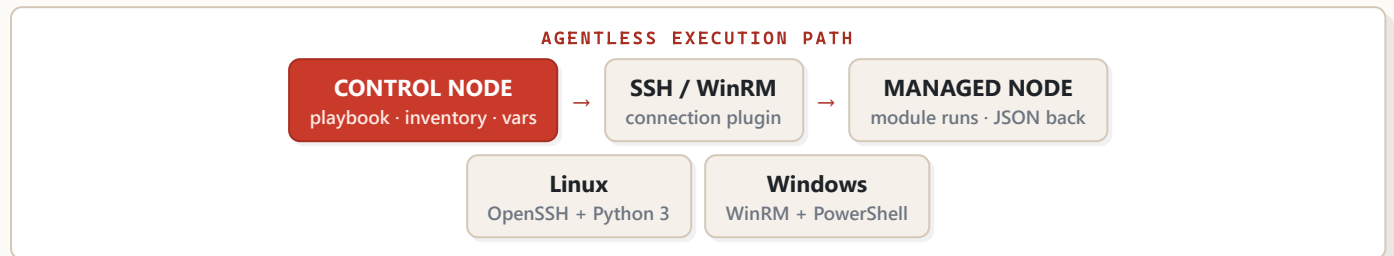
[NEXT]

We know **why** to automate. Next: **how** Ansible reaches machines — agentless architecture.

AGENTLESS – CONTROL TO MANAGED

▶ WHAT – Agentless architecture

Ansible requires no permanently running agent on managed nodes. The **control node** holds automation content and initiates connections; **managed nodes** receive module payloads and execute them.



▶ WHY – Why agentless

Problem: agent lifecycle (install, upgrade, cert rotation) is its own fleet problem. Agentless trades persistent presence for on-demand execution: simpler bootstrap, no agent fleet to patch, works over standard transports already present.

Tradeoff callout:

[WHY THIS EXISTS]

Agents give push-without-SSH but require ongoing fleet. Agentless gives immediate start but requires reachable transport + credentials per run. Choose agentless when SSH/WinRM already managed; choose agentless when pull/continuous drift correction required.

AGENTLESS – CONTROL TO MANAGED

▶ WHERE DOES IT FIT – Execution model

```
COMMAND → LOAD CONFIG (ansible.cfg) → LOAD INVENTORY → SELECT HOSTS → LOAD VARIABLES/FACTS → PROCESS
PLAY → EXECUTE TASK → RUN MODULE → RETURN RESULT → NEXT TASK
```

This lane is the visual anchor reused every chapter.

▶ HOW IT WORKS

- **Control node:** where ansible / ansible-playbook runs. Needs Python 3.11+, collections, inventory, ansible.cfg.
- **Managed node:** target host. Needs Python (most modules) or PowerShell (Windows), SSH/WinRM reachable, privilege path (become).
- **Transport:** connection plugin. Default ssh (Linux), winrm/psrp (Windows). Data sent: module payload + args; result returned as JSON.
- **Module execution:** module runs on managed node (or locally if delegate_to: localhost), reports changed, failed, skipped.

Config loading order (callout):

```
ansible.cfg precedence: ANSIBLE_CONFIG env → ./ansible.cfg → ~/.ansible.cfg →
/etc/ansible/ansible.cfg
```

▶ PRACTICAL – Verify control node

```
$ ansible --version
ansible [core 2.16.x] ...
```

Meaning: confirms control node install. Target: local. Internal: loads config, no inventory needed. Expected: version + config path. Failure: command not found → install ansible-core + ansible.

AGENTLESS – CONTROL TO MANAGED

► WSL SETUP – Control node on Windows

Windows cannot run Ansible natively. WSL2 + Ubuntu is the supported control node: a real Linux kernel where ansible-playbook, Python, and OpenSSH behave exactly like CI.

```
wsl --install -d Ubuntu          # Windows Terminal (admin): installs WSL2 + Ubuntu
sudo apt update && sudo apt install -y python3 python3-pip pipx openssh-client git
pipx install ansible             # ansible + ansible-playbook on PATH
pipx inject ansible ansible-lint
ansible --version                # expect: ansible [core 2.16.x], python 3.11+
```

Rules that prevent an entire class of failures:

- Keep the project **inside WSL home** (~/.ansible/), not /mnt/c/ — Windows mounts are slow and cannot hold Linux permissions (SSH keys at 0600 silently break there)
- Use VS Code with the **WSL Remote** extension so editors save with LF endings; CRLF line endings corrupt YAML and vault files
- Generate the SSH key **inside WSL** (ssh-keygen -t ed25519), copy the public half to servers with ssh-copy-id
- One .venv or pipx per toolset; never sudo pip install (breaks system Python)

```
ssh-keygen -t ed25519 -C "ansible-control"    # ~/.ssh/id_ed25519
ssh-copy-id ansible@10.0.1.11                 # test login without password prompt
ansible web -i inventory.ini -m ping          # first proof from WSL
```

► WHAT CAN FAIL

- UNREACHABLE → SSH key / WinRM creds / firewall
- MODULE FAILURE → Python missing on managed node
- become failure → sudoers / privilege escalation config

AGENTLESS – CONTROL TO MANAGED

Each maps to troubleshooting in PDF3 Ch 12.

▶ REAL-WORLD CONNECTION

Fleet of 200 VMs: one control node (CI runner) holds inventory + Vault. No agent deployed to 200 hosts; rotation handled by SSH key management.

▶ Interview check

Interview Why is Ansible called agentless?

Managed nodes need no permanent Ansible daemon. Control node connects via SSH/WinRM per task and executes modules on demand.

[NEXT]

Architecture tells *how* Ansible reaches machines. Next: *which* machines — inventory and targeting.

INVENTORY – WHICH MACHINES

▶ WHAT – Inventory is target declaration

Inventory lists managed nodes and organizes them into **groups**. Playbooks select from this list; ad hoc commands address it directly.



Static inventory example:

```

[web]
web-01 ansible_host=10.0.1.11
web-02 ansible_host=10.0.1.12

[db]
db-01 ansible_host=10.0.1.21

[prod:children]
web
db
  
```

Hierarchy:

```

INVENTORY
+-- groups (web, db, prod)
+-- hosts (web-01, web-02)
+-- group_vars / host_vars
+-- variables (ansible_user, ansible_become)
  
```

▶ WHY – Selection before declaration

Without inventory, automation has no audience. Inventory separates **which** from **what** — same playbook against web vs prod yields different blast radius.

INVENTORY – WHICH MACHINES

▶ HOW IT WORKS

- **Static:** INI or YAML file, committed to repo, versioned.
- **Dynamic:** plugin queries cloud (AWS EC2, Azure) at runtime — generates inventory, not checked in.
- **Patterns:** web, web[0], web:db, prod:&web, all:!db — composed at CLI/play hosts:.
- **Variables:** inventory vars flow into variable precedence (taught fully in PDF2 Ch 06) via group_vars/web.yml, host_vars/db-01.yml.

▶ PRACTICAL – Ad hoc proof (first execution)

Ad hoc = single task without playbook. Proves inventory + transport before YAML complexity.

COMMAND

```
ansible web -i inventory.ini -m ping
```

Meaning: run ping module against group web. Target: web-01, web-02. Internal: control node loads inventory, selects hosts, sends ping.py via SSH, returns JSON. Expected:

RESULT

```
web-01 | SUCCESS => { "changed": false, "ping": "pong" }
web-02 | SUCCESS => { "changed": false, "ping": "pong" }
```

Why it matters: proves selection, transport, Python on target in 5 seconds.

Common failure:

- UNREACHABLE → check ansible_user, ansible_host, key, ansible -m ping with -vvv
- changed: false surprise → ping is check-only; not a bug

Second ad hoc:

```
$ ansible web -i inventory.ini -a "uptime"
```

-a = command module args. Compare: -m ping vs -a "uptime" shows module vs raw command.

INVENTORY — WHICH MACHINES

▶ CONNECTING — SSH keys, users, IPs

Inventory names the targets; connection variables explain **how to reach them**. Four variables do 95% of the work:

```
[web]
web-01 ansible_host=10.0.1.11 ansible_user=ansible ansible_port=22
web-02 ansible_host=10.0.1.12 ansible_user=ansible

[web:vars]
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_become=true
```

- `ansible_host` — real IP/hostname (the inventory name stays a stable alias even when IPs change)
- `ansible_user` — SSH login user; create a dedicated ansible user, never automate as root over SSH
- `ansible_port` — non-default SSH port where hardened
- `ansible_ssh_private_key_file` — which key; one key per environment (`prod_ed25519` vs `staging_ed25519`)

Key bootstrap (once per server, from the control node):

```
ssh-keygen -t ed25519 -f ~/.ssh/prod_ed25519 # private stays 0600, never leaves control node
ssh-copy-id -i ~/.ssh/prod_ed25519 ansible@10.0.1.11
ssh -i ~/.ssh/prod_ed25519 ansible@10.0.1.11 # must log in with no password prompt
```

Behind a bastion, hop transparently instead of copying keys around:

INVENTORY – WHICH MACHINES

▶ CONNECTING – Bastions, first contact, trust

```
[web:vars]
ansible_ssh_common_args=-o ProxyJump=bastion.example.com
```

First-contact checklist (in order, stop at the first red):

```
ping -c2 10.0.1.11           # IP reachable at all?
ssh ansible@10.0.1.11        # key auth works by hand?
ansible web-01 -i inventory.ini -m ping    # Ansible transport works?
ansible web -i inventory.ini --list-hosts   # pattern selects who you think?
```

Host-key trust: first connection prompts Are you sure you want to continue connecting?. For known fleets, pre-seed ~/.ssh/known_hosts with ssh-keyscan; setting host_key_checking=false in ansible.cfg silences the prompt but also silences real machine-in-the-middle warnings — scope it to ephemeral lab inventories only.

▶ WHAT CAN FAIL

- Pattern selects zero hosts → hosts: web matches nothing if group misspelled → --list-hosts verify
- Wrong vars file location → group_vars/ must sit beside inventory file
- Dynamic inventory creds → plugin fails silently if cloud creds missing

▶ REAL-WORLD CONNECTION

Staging vs prod: same playbook, different inventories (inventories/staging/, inventories/prod/). Promotion = change -i path, not playbook.

INVENTORY – WHICH MACHINES

▶ LAB – Observe / Execute / Verify

Objective: prove inventory selection. Prereqs: inventory.ini above. Steps:

1. `ansible all -i inventory.ini --list-hosts` → Observe all targets
2. `ansible web -i inventory.ini -m ping` → Execute + Watch CHANGED vs SUCCESS
3. Verify pong. Explain: ping validates transport, not config.

Failure scenarios: wrong `ansible_host` → UNREACHABLE; fix → correct IP.

▶ Interview check

Interview Static vs dynamic inventory?

Static lists hosts in file; dynamic generates list at runtime from cloud API. Static for stable fleet, dynamic for elastic/autoscaling.

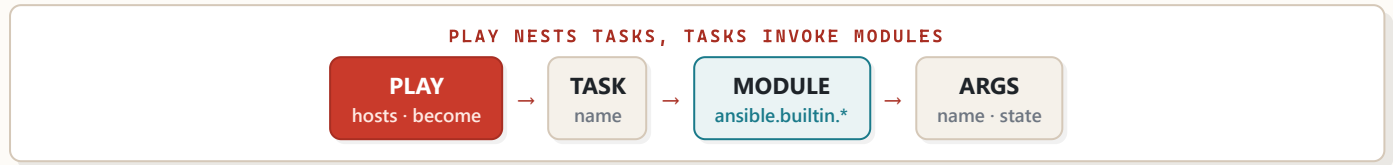
[NEXT]

We know **which** machines. Next: **what** state they should reach — playbooks.

PLAYBOOK – DESIRED STATE IN YAML

▶ WHAT – Playbook is desired-state document

A playbook maps **hosts** to **tasks** that invoke **modules** with **arguments**. It is YAML, declarative: describe end state, not step-by-step shell.



▶ WHY – From ad hoc to repeatable

Ad hoc proves one module once. Playbooks repeat, order, and audit multiple tasks. They are the unit committed to Git and triggered by CI.

ASPECT	AD HOC COMMAND	PLAYBOOK
Shape	One module call from the CLI	Plays and tasks in a YAML file
Repeat	Retype the command every time	Run the file again, identical result
Review	Shell history on one laptop	Git diff plus --check --diff
Handoff	Tell someone the command	Commit site.yml; CI runs it

PLAYBOOK – DESIRED STATE IN YAML

▶ YAML – Syntax before semantics

```
- name: Configure web server
  hosts: web
  become: true          # privilege escalation

  tasks:
    - name: Install nginx
      ansible.builtin.package:
        name: nginx
        state: present

    - name: Ensure nginx running
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```

Hierarchy:

```
PLAY (hosts: web, become: true)
+-- tasks
    +-- TASK (name: Install nginx)
        +-- MODULE (ansible.builtin.package, FQCN)
        +-- ARGUMENTS (name, state)
```

Rule: FQCN always (ansible.builtin.*) — short names obsolete. YAML whitespace matters; 2-space indent.

▶ HOW IT WORKS – Execution

```
PLAY (web) → TASK 1 (package) → SSH → managed node → check nginx absent → install → CHANGED
           → TASK 2 (service) → check stopped → start → CHANGED
           → RESULT per host → NEXT PLAY
```

Play = which hosts; Task = what module with which args; Module = implementation that converges state.

PLAYBOOK – DESIRED STATE IN YAML

▶ PRACTICAL – Run it

COMMAND

```
ansible-playbook -i inventory.ini site.yml
```

Meaning: execute playbook against inventory. Target: web group. Internal: load site.yml, select hosts, for each task send module, collect changed/ok/failed. Expected:

RESULT

```
PLAY [Configure web server] ***
TASK [Install nginx] *** changed: [web-01]
TASK [Ensure nginx running] *** changed: [web-01]
PLAY RECAP *** web-01 : ok=2 changed=2 failed=0
```

Why it matters: proves declarative convergence; re-run should yield ok=2 changed=0.

Common failure: YAML parse error → ERROR! Syntax Error at line; fix indent or missing :.

▶ WHAT CAN FAIL

- hosts: web matches zero → skipping: no hosts matched → verify with --list-hosts
- become: true without sudo → Failed to escalate → ansible_become_user / sudoers
- Module args typo → unsupported parameter → check ansible-doc ansible.builtin.package

PLAYBOOK – DESIRED STATE IN YAML

▶ REAL-WORLD CONNECTION

Team commits `site.yml` to `infra/` repo; reviewer sees `state: present` intent, not `apt-get install` imperative.

▶ LAB – Observe / Execute / Verify

Observe: `nginx` absent (`systemctl status nginx` fails). Execute: `ansible-playbook -i inventory.ini site.yml --check --diff` (dry run). Verify: `--diff` shows planned changes. Execute for real, verify ok with `changed=2`, re-run verify `changed=0`.

▶ Interview check

Interview Play vs task vs module?

Play selects hosts and holds tasks; task invokes one module with arguments; module does work on managed node and reports convergence.

[NEXT]

Playbooks declare state. Next: how modules enforce idempotency and report CHANGED vs OK.

MODULES AND IDEMPOTENCY

▶ WHAT – Module + ideal result

Module = unit of desired state. Takes args, checks current state, acts only if needed, returns JSON with changed, failed, msg.

FIRST RUN – CHANGED

```
Desired: nginx present ↓ Current: absent ↓ Install  
→ CHANGED
```

SECOND RUN – OK

```
Desired: nginx present ↓ Current: present ↓ No  
action → OK
```

Idempotency: same input applied twice → same end state, second run reports no change.

▶ WHY – Desired state ≠ re-run commands

Scripts re-run commands blindly. Modules compare desired vs current:

```
DESIRED STATE (state: present)  
↓  
CURRENT STATE (nginx absent? present?)  
↓  
COMPARE → CHANGE OR NO-OP → RESULT (CHANGED / OK)
```

Without this, automation cannot be safely repeated for drift correction or retries.

MODULES AND IDEMPOTENCY

▶ HOW IT WORKS – Visual

First run:

Desired: nginx present → Current: absent → Install → CHANGED

Second run:

Desired: nginx present → Current: present → No action → OK

Result states (always labeled with symbol+color, not color alone):

- OK (green ●) — converged, no change
- CHANGED (blue ●) — converged, did change
- FAILED (red □□) — did not converge
- SKIPPED (gray ○) — conditional skipped
- UNREACHABLE (amber □□) — transport failed

DESIRED VS CURRENT DECIDES EVERYTHING

DESIRED



CURRENT



COMPARE



CHANGE or NO-OP

MODULES AND IDEMPOTENCY

▶ PRACTICAL – Observe idempotency

```
- name: Idempotency demo
  hosts: web
  tasks:
    - name: Create app directory
      ansible.builtin.file:
        path: /opt/app
        state: directory
        mode: '0755'
```

First run: changed: [web-01]. Second run: ok: [web-01]. Proof of desired-state model.

Check and diff modes:

```
$ ansible-playbook -i inventory.ini site.yml --check --diff
```

--check dry-runs, --diff shows planned file/service changes. Use for approval gates.

▶ WHAT CAN FAIL – Common misreads

- OK misread as "nothing happened" → actually "already converged" (GOTCHA callout below)
- Module not idempotent by misuse → `command: apt-get install` always CHANGED vs `package: converges` → use `package/file/template` not `command/shell` where module exists
- `changed_when` abuse → masks real change signal

[GOTCHA]

"OK" does not mean Ansible did nothing meaningful. It means desired state already holds. Always read result state pair: ok vs changed count.

MODULES AND IDEMPOTENCY

▶ COMMAND + RESULT PATTERN

COMMAND

```
ansible web -i inventory.ini -m ansible.builtin.file -a "path=/opt/app state=directory mode=0755"
```

RESULT

```
web-01 | CHANGED => { "changed": true, "path": "/opt/app", "state": "directory" }
```

WHAT HAPPENED: file module ensured directory exists, created it.

WHY IT MATTERS: proves CHANGED on creation, OK on repeat.

COMMON FAILURE: path parent missing → use state=directory (creates parents), mode typo → permission drift.

▶ COMPARISON – Module vs shell

ASPECT	MODULE (FILE, PACKAGE)	COMMAND/SHELL
Intent	desired state	imperative command
Idempotency	built-in compare	always CHANGED unless creates/removes
Check mode	supported	skipped
Audit	diff friendly	opaque

▶ REAL-WORLD CONNECTION

Patch fleet: re-run same playbook weekly → CHANGED only where drift occurred. changed count becomes drift metric fed to observability.

▶ Interview check

Interview What makes a module idempotent?

It compares desired vs current state and acts only when different, reporting CHANGED when it acted and OK when already converged.

[NEXT]

Mechan mechanics mastered in PDF1. Next PDF: data — variables, facts, registers — that make modules vary per host.

CHEATSHEET – FQCN AND FAILURES

A FQCN – Use Always

LEGACY	FQCN
yum, apt	ansible.builtin.package
template	ansible.builtin.template
service	ansible.builtin.service
file	ansible.builtin.file

B Failure Index

SYMPTOM	FIX
UNREACHABLE	SSH key / ansible_host
Failed to escalate	become + sudoers
YAML error	ansible-lint
OK misread	already converged

● OK – converged, no change

● CHANGED – converged, did change

❌ FAILED – did not converge

END FOUNDATIONS · VOL 1

VOL 2 – PLAYBOOK ENGINEERING

Variables, logic, templates and roles: one playbook behaving differently per host.